#15/02/2026

Software development with Agentic AI

A few years ago, being a developer meant mastering syntax, writing functions from scratch, and controlling every line of code. However, today, the paradigm has changed drastically with the arrival of agenic architectures and development model.

This article presents an overview for the AI-enhanced software development model describing how agentic AI can be strategically incorporated into the Software Development Life-Cycle (SDLC).

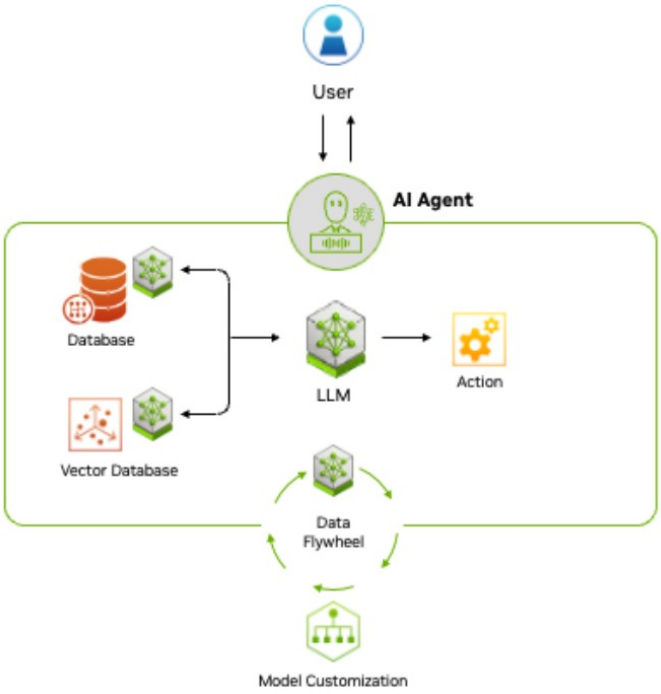
Author of the article: *Iker Elg, Alzaa*

#SoftwareDevelopment #AgenticAI

https://github.com/kraftcoding

Agentic development is a transformative software approach where AI agents actively plan, write, test, and ship production-grade code, moving beyond simple autocomplete to act as collaborative partners.

It involves AI managing start-to-end tasks like refactoring, documenting, and debugging. This methodology increases development velocity by enabling AI to handle complex tasks while human developers focus on system architecture and review.



Agentic AI uses sophisticated reasoning and iterative planning to autonomously solve complex, multi-step problems.

To understand how this approach is applied, it is important to understand the differences between imperative and declarative programming.

Coding efficiently

Declarative programming plays a significant role in agentic software development by allowing AI agents to focus on specifying what needs to be achieved, rather than how to achieve it.

To understand the differences between imperative and declarative programming think about imperative programming as a way of describing how things work, and declarative programming as a way of describing what you want to achieve.

Let's move into a JavaScript example. Here we will write a simple function that, given an array of lowercase strings, returns an array with the same strings in uppercase:

```
toUpperCase(['foo', 'bar']) // ['FOO',
'BAR']
```

An imperative function to solve the problem would be implemented as follows:

```
const toUpperCase = input ⇒ {
const output = []
 for (let i = 0; i < input.length; i++) {
   output.push(input[i].toUpperCase())
 }
 return output
}
```

A declarative solution in Type Script would be as follows:

```
const toUpperCase = input ⇒
input.map(value ⇒ value.toUpperCase())
```

In declarative programming, developers define the desired outcome, and the AI agent figures out the implementation details.

This synergy between declarative programming and agentic development enables a more efficient and scalable software development.

AI-powered assistant

On the another hand AI-powered assistant offers Copilot functions tailored to the specific

needs of each environment—focusing on application development or data analysis tools and environments.

Declarative programming together with AI assisted copilot, enables:

- Faster development: AI agents can generate code and optimize solutions based on the desired outcome adapted to each environment.
- Improved flexibility: Changes in requirements are easier to implement, as the AI agent adapts the implementation.
- Reduced complexity: Developers focus on defining the problem, while AI handles the underlying complexity.

For software development, this means AI systems that don't just assist developers but can independently perform complex development tasks while *maintaining awareness of the broader project context*.

Challenges and opportunities

By distributing specialized AI agents throughout the development lifecycle, organizations can achieve unprecedented levels of productivity, quality, and innovation.

However, successful implementation requires thoughtful architecture, clear human oversight mechanisms, and gradual adoption that builds trust and allows both human developers and AI systems to learn how to collaborate effectively.